

并发任务编排的实现与设计

以 Java CompletableFuture 为例

尺规 @oldratlee

2025-10

0.0 通过 并发编排 缩短RT 的例子

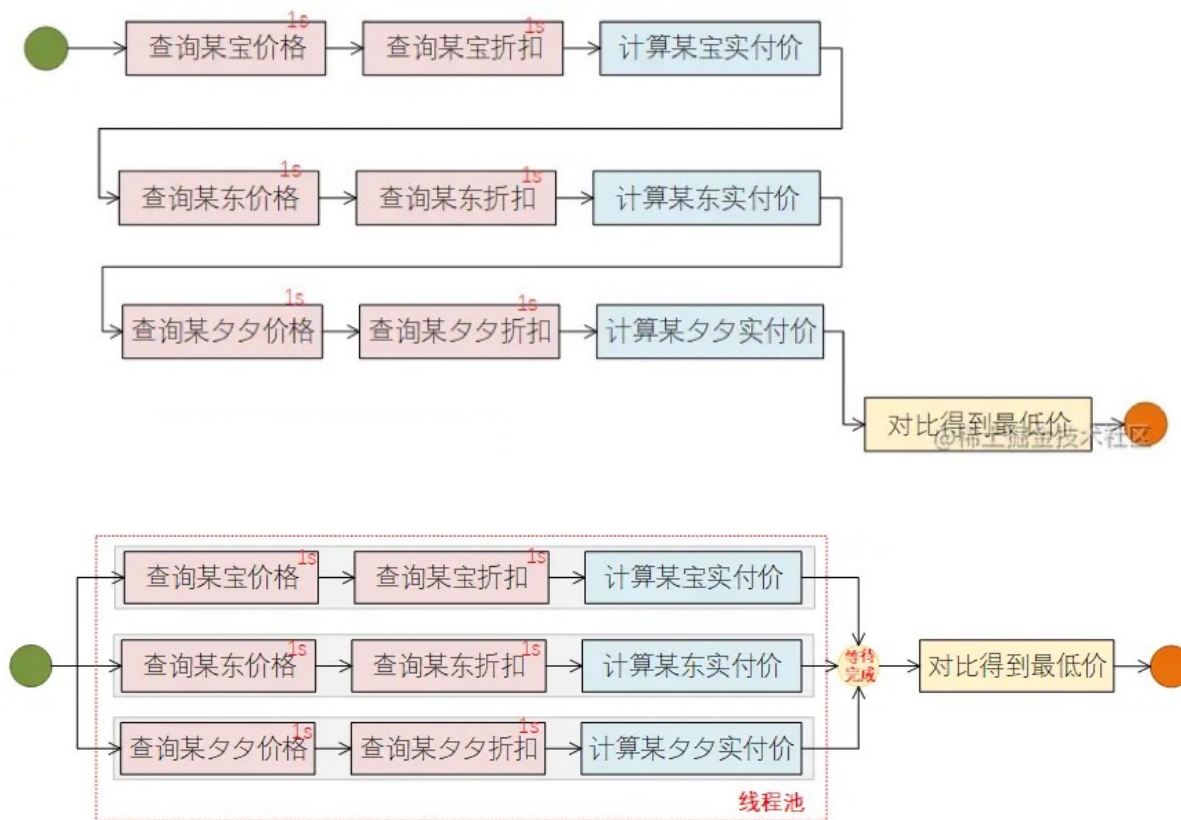
串行

等待 6s



并行

等待 2s

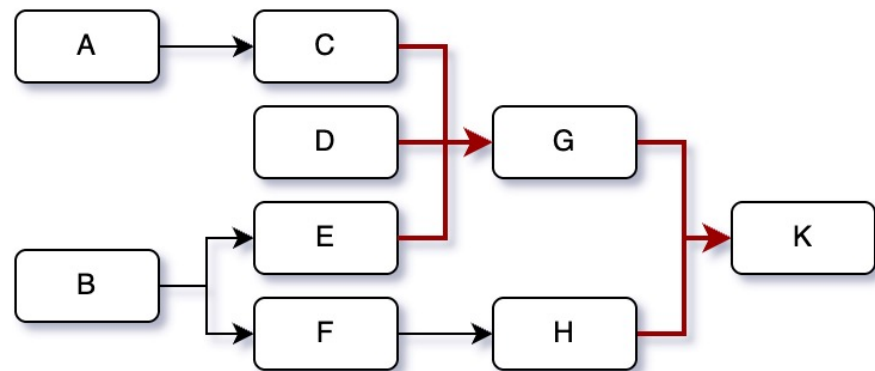


图片来源: Java 基于 CompletableFuture 的流水线并行处理深度实践
juejin.cn/post/7124124854747398175

0.1 关于编排

这里讨论的 编排 指：

- ▶ 协调**多个**(并发/异步)**任务**
 - ▶ 任务可以有 执行逻辑 与 执行结果
- ▶ 然后**触发**其它的任务
 - ▶ 可以有不同 触发/编排策略
- ▶ 简单编排，即： **一个**任务 触发 其它(不同)任务， 不作讨论
 - ▶ 因为 理解与使用 都很简单自然
- ▶ 不同说法如「异步（任务）编排」「并发（任务）编排」等， 不作区分



```
CompletableFuture<String> cf1 = CF.supplyAsync(() -> "1");  
CompletableFuture<String> cf2 = CF.supplyAsync(() -> {throw new Exception("2");});  
CompletableFuture<String> cf3 = CF.supplyAsync(() -> "3");
```

```
CompletableFuture<Void> all = CF.allOf(cf1, cf2, cf3);
```

```
all.join();  
println(cf1.join() + cf2.join() + cf3.join());
```

0.2 关于编排策略

标准库 `CompletableFuture` 提供了两种策略：

- ▶ `All(Complete)`
- ▶ `Any(Complete)`

```
// All(Complete)
CompletableFuture<Void> all = CF.allOf(cf1, cf2, cf3);
CompletableFuture<String> both = cf1.thenCombine(cf2, (x, y) -> x + y);
...

// Any(Complete)
CompletableFuture<String> any = CF.anyOf(cf1, cf2, cf3);
CompletableFuture<String> either = cf1.applyToEither(cf2, x -> x + "!");
...
```

▶ AllComplete vs AllFailFast

- ▶ `AllComplete` 会等待所有输入任务运行完成；即使有任务失败了也要等待后续任务都运行完成，再返回一个失败的结果。
- ▶ 对于业务逻辑来说，**失败且继续等待**的策略，**降低了业务响应性**。
- ▶ 业务需要的是，当有输入任务失败了则**快速失败不再做于事无补的等待**。

▶ AnyComplete vs AnySuccess

- ▶ `AnyComplete` 返回首个完成的任务，不会等待后续没有完成的任务；即使**首个完成的任务是失败的**，也会返回这个失败任务的结果。
- ▶ 业务逻辑一般需要的是**首个成功任务的结果**，而不是首个完成但**失败**任务的结果。

关于 `AllFailFast` / `AnySuccess` 的更多介绍与讨论 参见：

`CompletableFuture` 如何实现异步任务编排中最常用的模式 —— 快速失败
juejin.cn/post/7420597224546091059

1.A 如何实现 AllFailFast ?

用什么并发工具都可以，比如使用

任务执行：

- ▶ `Thread` (new / Run)
- ▶ `ThreadPool` (execute / submit)
- ▶ `CompletableFuture`
- ▶ ...

任务协调：

- ▶ `CountDownLatch`
- ▶ `Future` (f.get / f.cancel)
- ▶ `CompletableFuture`
- ▶ ...

API 与实现 可以与工具 CF 没有关系；实现 AllFailFast 功能效果即可

对于基于 CF 的实现，这个方法签名会是：

```
public static <T> CompletableFuture<List<T>>  
allResultsFailFastOf(CompletableFuture<? extends T>... cfs);
```

- ▶ AllFailFast
 - ▶ 当有输入 CF 失败了，则快速失败
- ▶ AllComplete
 - ▶ 等待所有输入 CF 运行完成
 - ▶ 即使有 CF 失败了也要等待后续CF都运行完成
- ▶ 两者都是，仅当所有的输入都成功时，才返回成功的结果



还有 如何实现 AnySuccess ?

类似，讨论一者就好

1.B 如何通过 CF.allOf + CF.anyOf 实现 AllFailFast ?

实现上可以假定 CF 有了方法 **AllResultsOf**，会返回多个输入任务的结果 (List)：

```
public static <T> CompletableFuture<List<T>>  
allResultsOf(CompletableFuture<? extends T>... cfs);
```

而不像 CF.allOf 方法没有返回任务的执行结果 (Void)：

```
public static CompletableFuture<Void>  
allOf(CompletableFuture<?>... cfs);
```



先自己想一想? 🤔

- ▶ AllFailFast
 - ▶ 当有输入 CF 失败了，则快速失败
- ▶ AllComplete
 - ▶ 等待所有输入 CF 运行完成
 - ▶ 即使有 CF 失败了也要等待后续CF都运行完成
- ▶ 两者都是，仅当所有的输入都成功时，才返回成功的结果

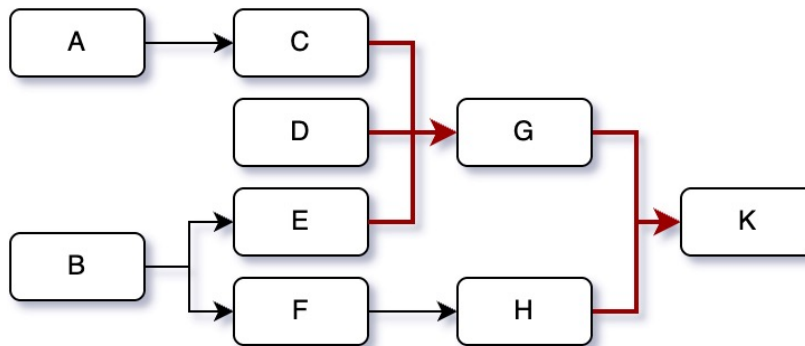
参见 cffu 库的代码实现：
[CompletableFutureUtils.java#allFailFastOf0\(\)](#)

2. 还有 哪些 编排策略？

前面提到了 4种编排策略：

- ▶ AllComplete vs AllFailFast
- ▶ AnyComplete vs AnySuccess

还有哪些？ 只有哪些？



先自己想一想？ 🤔

参见 cffu 库的文档：[高效灵活的并发执行策略](#)

3. 并发编排实现设计的注意点（以 AllFailFast 为例）

- ▶ **即时**触发，保证响应性
 - ▶ 这是要正确实现的时序功能，否则是 Bug
- ▶ 不吞**异常**，方便排查业务问题
 - ▶ 当多个输入抛出**多个异常**时，这些异常**至多只能有一个**能通过返回**反馈给业务**
 - ▶ 其它没返回的异常**不能被默默地吞掉**、要报告如打日志出来，否则**影响业务问题的排查**
- ▶ 尽快释放**内存**，避免**内存泄漏**
 - ▶ AllFailFast 会返回(持有) 多个输入任务的结果
 - ▶ 当有任务失败时，所有结果都不再需要了，应该立即释放所有结果对象
 - ▶ 以避免 因存在长时间运行的任务，而持有其它任务的(大)结果对象
- ▶ 当输入**0个**任务时，如何返回、返回什么？
 - ▶ 关注好 平凡情况、边界 Case；设计一致性、实现可靠性

这些注意点可以看看 cffu 库的文档 或 源码，有讨论与答案：

- ▶ [github.com/foldright/cffu/.../docs/README_CN.md](https://github.com/foldright/cffu/blob/master/docs/README_CN.md)
- ▶ [github.com/foldright/cffu/.../cffu-core/src/main/java/io/foldright/cffu2/CompletableFutureUtils.java](https://github.com/foldright/cffu/blob/master/cffu-core/src/main/java/io/foldright/cffu2/CompletableFutureUtils.java)

参考资料

- ▶ Java 基于 CompletableFuture 的流水线并行处理深度实践
juejin.cn/post/7124124854747398175
- ▶ CompletableFuture 如何实现异步任务编排中最常用的模式——快速失败
juejin.cn/post/7420597224546091059
- ▶ 🐼 Java **CompletableFuture-Fu** ("CF-Fu", pronounced "Shifu") is a tiny 0-dependency library that improves the CompletableFuture (CF) usage experience and reduces misuse, enabling **more convenient, efficient, and safe use** of CF in your application. 😊🚀🦺
github.com/foldright/cffu

Thanks

&

QA

